Does Scale Make Long-Context Learning Cheaper?

Vatsal Baherwani, Ammar Ali, Abhishek Muhuri

Problem Statement & Motivation

Training on long contexts is expensive, and models still struggle to use information far back in their context (Liu et al., 2023). We ask whether larger models learn long-context tasks more efficiently, and how the answer changes with context length. Larger models typically need fewer steps to learn a task, but each step costs more compute. The practical question is therefore which model size reaches a long-context capability with the least total compute (FLOPs), and whether that optimal size grows with context length. We also ask which part of scale matters: attention heads, head dimension, depth, or width.

We study this question with small transformers on synthetic retrieval tasks, where we control context length and task difficulty exactly. Our findings will give concrete guidance on how to size and shape a model for long-context training under a fixed compute budget. As a stretch goal, we test whether distilling from a larger model or using a short-to-long curriculum reduces this compute further.

Related Work

Baherwani et al. (2026) show that capabilities such as in-context learning emerge abruptly and at random times during training, when the model learns a sparse attention pattern. That work also finds that sparse patterns become harder to learn as context length grows, and that larger models and more attention heads learn them earlier on average. However, it studies context length with a fixed model size and does not measure compute efficiency across scales. Scaling laws (Kaplan et al., 2020; Hoffmann et al., 2022) describe how loss improves with compute, but they do not describe when a specific capability appears or how context length changes the compute-optimal model size.

Our tasks come from the long-context evaluation literature. Multi-query associative recall (MQAR) separates architectures by their ability to recall information from context (Arora et al., 2023). Needle-in-a-haystack tests retrieval of a single fact hidden in long filler text (Kamradt, 2023; Hsieh et al., 2024). Both tasks rely on induction-style attention patterns (Olsson et al., 2022), which makes them a natural testbed for how attention learning scales with context.

For our stretch goals, knowledge distillation trains a small student on the outputs of a larger teacher (Hinton et al., 2015). Busbridge et al. (2025) show that distillation beats direct training only when the teacher's cost is excluded or amortized over many students, so we report both accountings. Curricula that increase sequence length during training improve the stability and efficiency of pretraining (Li et al., 2022), and most long-context models are trained at short

context before being extended (Xiong et al., 2023). We test whether these interventions specifically speed up learning a long-context capability.

System Design & Scope

We train decoder-only transformers from scratch with next-token prediction and rotary position embeddings. The main sweep uses five model sizes, roughly 1M, 4M, 12M, 25M, and 60M non-embedding parameters, which scale depth and width together. We train each model at context lengths of 256, 512, 1024, and 2048 tokens.

We use two synthetic tasks that separate two sources of long-context difficulty. In needle-in-a-haystack, each sequence contains a single key-value pair at a random position among random filler tokens, and the model must output the value when queried with the key at the end. The retrieval problem stays fixed while the filler grows, so this task measures the cost of length alone. In MQAR, each sequence contains a list of key-value pairs followed by queries for a subset of the keys. We scale the number of pairs with context length, so this task measures the cost of length together with the number of items the model must track. We draw a fresh random sequence for every example, so the models cannot memorize the training data.

To study which part of scale matters, we fix a model of about 12M parameters and run three controlled sweeps. First, we vary the number of heads from 2 to 32 at fixed width, setting head dimension to width divided by heads. Second, we fix the number of heads and vary head dimension from 8 to 64, decoupling the attention dimension from the model width. Third, we trade depth against width from 2 to 16 layers at a matched parameter count. We run these sweeps at context lengths 512 and 2048.

We are scoping out natural language data and chess. In chess, longer games are also harder positions, so length and difficulty would be confounded. Synthetic tasks let us vary length while holding difficulty fixed.

Evaluation Pipeline

Our main metric is the compute needed to reach 90% accuracy on held-out sequences. For needle-in-a-haystack, accuracy is exact match on the value token. For MQAR, accuracy is the fraction of queries answered correctly. We evaluate every 250 steps and record the first step after which accuracy stays above 90% for three consecutive evaluations, so a single noisy evaluation does not count as learning the task. We convert steps to FLOPs with an analytic count that includes both the parameter term and the attention term, since attention cost grows quadratically with context and dominates for small models at long context. We also bucket needle-in-a-haystack accuracy by the needle's position, to test whether our models show the lost-in-the-middle effect that Liu et al. (2023) observe in LLMs.

Prior work shows that capability emergence time varies across random seeds (Baherwani et al., 2026), so we train 5 seeds per setting and report the median and range of FLOPs to threshold.

Runs stop once they meet this criterion or after 20k steps. We report the fraction of seeds that reach the threshold, so runs that fail are visible rather than silently dropped. We use a constant learning rate after warmup, so the time to threshold does not depend on the step cap. We also tune the learning rate separately for each model size, so that differences between sizes do not reflect a learning rate that suits one size better than another.

We consider the project successful if we can state, for each task, whether the compute-optimal model size changes with context length, including a clear absence of a trend. For the stretch goal, success means at least one context length where we can state whether distillation beats direct training on total compute.

Experimental Plan

The main sweep covers 5 sizes, 4 context lengths, 2 tasks, and 5 seeds, for 200 runs. The architecture sweeps add about 12 configurations at 2 context lengths, 2 tasks, and 5 seeds, for 240 runs. Our analytic estimate puts the largest single run (60M parameters at 2048 tokens) at about 1.3 H100 hours. All of our models fit on a single GPU, and we can pack several small runs onto one GPU. Including learning rate tuning, stretch goals, and a 4x margin for low GPU utilization and debugging, we budget about 600 GPU hours on NYU's Torch cluster.

For distillation, we reuse checkpoints of the 60M models from the main sweep as teachers, so their training is already paid for. We take teacher checkpoints from before, during, and after the teacher reaches the accuracy threshold, to test whether a briefly trained teacher can already speed up a student. We train the 1M, 4M, and 12M students with a mix of cross-entropy and KL divergence to the teacher's output distribution, at context lengths 512 and 2048. We report FLOPs to threshold under two accountings. The first counts only the student's training. The second also counts the teacher's training up to the checkpoint and the teacher's forward passes during distillation. The break-even point is the number of students the teacher must serve before distillation beats direct training. For the curriculum, we train the same three sizes at 256 tokens and then switch to 2048 tokens, and compare total FLOPs against training at 2048 from the start.

Our schedule spans seven weeks. In week 1 we build the data generators, FLOP counter, and training loop, and we run a pilot to calibrate task difficulty. We run the main sweep in weeks 2 and 3 and the architecture sweeps in week 4. We spend weeks 5 and 6 on the stretch goals and week 7 on analysis and writing.

Investigation & Iteration Plan

Our central plot shows FLOPs to threshold against model size, with one curve per context length. If larger models help more at longer contexts, the compute-optimal size should shift right as context grows. Comparing the two tasks then tells us what scale buys. If larger models help much more on MQAR than on needle-in-a-haystack, scale mainly provides capacity to track many items, rather than the ability to search a long context.

When results are unexpected, we inspect attention maps. Both tasks define exactly which token the model must attend to, so we track the attention mass each head places on the correct key over training. Following Baherwani et al. (2026), we test whether the jump in accuracy coincides with the model learning this retrieval pattern. We then check whether larger models learn this pattern earlier or simply use it more reliably. The architecture sweeps test the prior finding that more heads help up to a minimum head dimension, now as a function of context length.

We will iterate on task difficulty after the pilot. If every model reaches the threshold within a few hundred steps, we increase the vocabulary size or the number of MQAR pairs. If no model learns the task at 2048 tokens, we lower the threshold or extend the step cap for that setting. If logit distillation does not help the student, we add a loss that matches student attention maps to the teacher's retrieval heads, since the capability depends on learning a sparse attention pattern.

Challenges & Mitigation

The main risk is poorly calibrated tasks, which we address with the pilot in week 1. A second risk is high variance across seeds, which could hide differences between model sizes. We mitigate this with 5 seeds and by reporting ranges, and we can add seeds for the settings where size curves overlap. A third risk is an unfair comparison across sizes, which we address by tuning the learning rate for each size and counting FLOPs rather than steps. Finally, distillation may be slow to get working, so the curriculum serves as a cheaper fallback for the stretch goal.

Expected Learning

We expect to learn how compute-optimal model size depends on context length for a specific capability, rather than for overall loss. We will also learn whether the benefit of scale comes from heads, head dimension, depth, or width, and whether that answer changes at long context. Practically, we will gain experience running controlled scaling sweeps, counting FLOPs correctly, and analyzing training dynamics across many seeds.

Future Directions

The natural next step is to test whether our trends hold on natural language long-context benchmarks, or on chess with position difficulty explicitly controlled. Prior work also shows that architectures other than attention, such as MLP-Mixer and recurrent models, behave differently on these tasks (Arora et al., 2023; Baherwani et al., 2026), so comparing their compute-optimal scaling against transformers is a direct extension.

References

Arora, S., et al. (2023). Zoology: Measuring and improving recall in efficient language models. arXiv:2312.04927.

Baherwani, V., Chen, Z., Qiu, S., Wilson, A. G., and Izmailov, P. (2026). Emergent capabilities arise randomly from learning sparse attention patterns. arXiv:2606.25010.

Busbridge, D., et al. (2025). Distillation scaling laws. arXiv:2502.08606.

Hinton, G., Vinyals, O., and Dean, J. (2015). Distilling the knowledge in a neural network. arXiv:1503.02531.

Hoffmann, J., et al. (2022). Training compute-optimal large language models. arXiv:2203.15556.

Hsieh, C.-P., et al. (2024). RULER: What's the real context size of your long-context language models? arXiv:2404.06654.

Kamradt, G. (2023). Needle in a haystack: Pressure testing LLMs. github.com/gkamradt/LLMTest_NeedleInAHaystack.

Kaplan, J., et al. (2020). Scaling laws for neural language models. arXiv:2001.08361.

Li, C., Zhang, M., and He, Y. (2022). The stability-efficiency dilemma: Investigating sequence length warmup for training GPT models. arXiv:2108.06084.

Liu, N. F., et al. (2023). Lost in the middle: How language models use long contexts. arXiv:2307.03172.

Olsson, C., et al. (2022). In-context learning and induction heads. arXiv:2209.11895.

Xiong, W., et al. (2023). Effective long-context scaling of foundation models. arXiv:2309.16039.